

Ressource Développement Web S3

TD n°5 : encapsulation, gestion de listes

Tout le code produit doit être conforme au standard PSR-1.

Les types des arguments et les types de retour doivent être précisés.

On souhaite créer une application web pour un service de diffusion de musique en ligne. L'application manipule donc principalement des pistes audio de différents types (piste d'album, podcast) ainsi que des listes de pistes (albums, playlists, liste d'attente).

Exercice 1

Faites une copie de toutes les classes et programme du TD 4 dans un nouveau répertoire.

Reprendre les classes `AudioTrack`, `AlbumTrack` et `PodcastTrack` :

- transformer tous les attributs public en private ou protected selon le cas,
- tester votre programme principal qui crée des objets et valorise les attributs : fonctionne-t-il encore ? expliquer les erreurs.
- remédier à ces erreurs :
 - en ajoutant un getter générique, ou en construisant la méthode magique `__get()` (vérifier le profil exact dans la documentation officielle PHP). Au cas où la propriété accédée n'existe pas, ces deux méthodes déclenchent une `Exception` avec le message d'erreur `"invalid property : <attrname>"`.
 - En ajoutant les setter nécessaires ; on choisi de rendre non modifiables les propriétés initialisées par le constructeur de chaque classe. Par conséquence, pas de setter pour ces attributs.
- Adaptez votre programme principal pour qu'il fonctionne de nouveau et accède aux propriétés des objets.

Exercice 2

On souhaite maintenant faire des vérifications lors des accès aux propriétés de la classe `AudioTrack`. En cas d'erreur on déclenchera une exception. Afin de catégoriser ces erreurs, on crée une sous-classe `Exception` dédiée pour chaque cas.

- propriété inconnue : `InvalidPropertyNameException`
- mauvaise valeur de propriété, à appliquer à `$duree` pour une valeur `< 0` : `InvalidPropertyValueException`.

Dans le programme principal, ajoutez un try-catch pour attraper les exceptions potentielles. En cas d'erreur, afficher le message contenu dans l'exception et la trace d'exécution du script.

Exercice 3

On souhaite maintenant gérer des listes de pistes audio, qui correspondent à deux utilisations : les albums et les playlist.

Une liste audio (classe `AudioList`) est décrite par un nom, un nombre de pistes, une durée totale et un tableau contenant les pistes constituant la liste.

- créer la classe `AudioList` en programmant un constructeur qui reçoit en paramètre le nom de la liste et un tableau optionnel de piste qui prendra la valeur `[]` s'il est omis. Le constructeur initialise les propriétés, notamment en calculant le nombre de pistes et la durée totale. La classe implante le getter magique `__get()` pour accéder au valeur des propriétés mais ne permet pas leur modification.
- Créer une sous-classe pour représenter les albums. Un album est décrit par les propriétés

héritées ainsi que un artiste de l'album et une date de sortie. Le constructeur d'album impose la présence du tableau contenant la liste des pistes de l'album. Cette liste ne sera plus modifiable. La classe album définit des setters permettant de valoriser uniquement l'artiste de l'album et la date de sortie de l'album.

- Créer une sous-classe pour représenter les Playlists. Une playlist est une liste audio pour laquelle on peut ajouter ou retirer une piste quelconque à tout moment, ou ajouter une liste de piste en une fois.
 - créer la sous-classe et programmer la méthode pour ajouter une piste quelconque ; il faut bien sûr mettre à jour le nombre de piste et la durée totale,
 - programmer la méthode pour supprimer une piste ; elle reçoit en paramètre l'indice de la piste à supprimer,
 - programmer la méthode permettant d'ajouter une liste de pistes en une opération ; elle reçoit en paramètre un tableau de pistes à ajouter aux pistes déjà présentes ; essayez de trouver une solution simple pour gérer les éventuels doublons.

Exercice 4

On souhaite maintenant pouvoir afficher les listes de pistes en HTML. Pour simplifier la programmation, on affichera de manière identique les albums et les playlists. Il suffit donc de créer un seul renderer pour les AudioList.

Programmer la classe `AudioListRenderer` qui implante `Renderer`. Le constructeur reçoit une `AudioList` en paramètre.

La méthode `render()` n'utilise pas le sélecteur et produit toujours le même affichage : le nom de la playlist, suivi de chaque piste en mode compact, et fin de liste le nombre de pistes et la durée totale.

Exercice 5 : namespaces

Ajouter des namespace dans vos classes. Les namespace à utiliser seront structurés de la manière suivante :

- `iutnc\deefy\audio\tracks` : pour les classes correspondants à des pistes audio,
- `iutnc\deefy\audio\lists` : pour les classes correspondants à des listes de pistes audio,
- `iutnc\deefy\render` : pour les classes correspondants au rendu html,
- `iutnc\deefy\exception` : pour les classes correspondants à des exceptions.

Ajouter ces namespaces dans chacune des classes, et adaptez votre programme principal pour qu'il manipule les classes avec leur nom complet. Au besoin, placez des `use` pour alléger les notations.

Déplacez maintenant tous vos fichiers contenant des définitions de classe ou interface vers le répertoire `src/classes`. Quel est l'impact sur votre programme principal ?